TRACKORA

NestJS API + PostgreSQL + Prisma

Backend Development Plan

Goal: provide one secure API and database for both the Flutter mobile app and Next.js admin website. The backend is the source of truth for permissions, visits, attendance, approvals, targets and audit history.

Layer	Technology	Responsibility
API	NestJS + TypeScript	REST endpoints, validation, business rules and authorization
Database	PostgreSQL	Persistent relational data, constraints, transactions and reports
ORM/migrations	Prisma	Typed queries, schema and versioned migrations
Authentication	Managed identity provider or secure auth implementation	Identity verification, session/token lifecycle and account state
Files	Private object storage	Visit photos/audio with controlled access
Notifications	FCM + server-side jobs	Push delivery, retries and scheduled reminders
Operations	Tests, logs, monitoring and backups	Reliability, diagnostics and recovery

Backend rule: never trust the frontend to authorize sensitive actions or calculate authoritative approval/target results. Enforce access rules on every request.

1. Backend architecture

```
apps/api/
■■■ src/
■   ■■■ common/          # guards, pipes, filters, errors
■   ■■■ config/          # validated environment configuration
■   ■■■ auth/
■   ■■■ companies/
■   ■■■ users/
■   ■■■ teams/
■   ■■■ customers/
■   ■■■ contacts/
■   ■■■ attendance/
■   ■■■ visits/
■   ■■■ attachments/
■   ■■■ approvals/
■   ■■■ targets/
■   ■■■ leaderboard/
■   ■■■ notifications/
■   ■■■ reports/
■   ■■■ audit/
■■■ prisma/
    ■■■ schema.prisma
    ■■■ migrations/
```

Start with a modular monolith: one deployable API with clearly separated modules. Avoid microservices until actual scale or operational requirements justify them.

2. Core database entities

Entity	Purpose and relationships
Company	Tenant boundary, timezone and policy settings
User	Company, role, status, manager/team membership and identity-provider subject
Team	Team details and assigned manager
Customer / Contact	Business record and people associated with it
Visit	Employee, customer, visit date/type, notes, GPS evidence, status and timestamps
VisitContact	Join table for multiple contacts linked to a visit
Attendance	Employee, work date, check-in/out timestamps and location evidence
Attachment	Private object key, media type, size, upload status and visit relationship
Target	Metric, period, goal, scope and effective dates
Approval	Visit, reviewer, decision, reason and decision timestamp
AuditLog	Actor, action, entity, timestamp and relevant before/after values
Notification	Recipient, event, delivery state, retry count and timestamps
LeaderboardSnapshot	Period, rule version and historical ranking where required

Data modelling rules

• Use consistent IDs, foreign keys, indexes and database constraints.

• Include company/tenant ownership on company-scoped records and enforce it in every query.

• Store timestamps in UTC; use company timezone for daily attendance and reports.

• Use transactions for approval transitions and updates affecting multiple records.

• Store media in private object storage and metadata in PostgreSQL.

• Use migrations; do not manually change production tables without versioning.

3. Proposed API surface

Method and path	Purpose
POST /auth/login or identity callback	Establish session, depending on identity provider
GET /users/me	Current user profile and role context
POST /attendance/check-in	Record check-in with server timestamp and submitted location evidence
POST /attendance/check-out	Record check-out if enabled by policy
GET /attendance/me?date=YYYY-MM-DD	Personal attendance for a date
POST /visits	Create a visit draft or submission per agreed contract
GET /visits/me?date=YYYY-MM-DD	Personal visit list with filters/pagination
GET /visits/:id	Authorized visit details
POST /visits/:id/submit	Submit draft for review
GET /admin/visits?status=PENDING	Authorized review queue
POST /admin/visits/:id/approve	Approve pending visit
POST /admin/visits/:id/reject	Reject pending visit with reason
GET /admin/team/:id/daily?date=YYYY-MM-DD	Historical team activity
GET /targets/me	Assigned targets and progress
GET /leaderboard?period=monthly	Ranking for agreed period
GET /admin/reports	Authorized filtered reports

These are proposed endpoints, not existing functionality. Finalize payloads, response schemas, pagination and error format with the frontend developer.

4. Visit approval state machine

```
DRAFT -> PENDING -> APPROVED
                 \-> REJECTED -> EDIT / RESUBMIT (if policy allows)

- Only authorized roles can approve/reject.
- Rejection requires a reason.
- Only one competing decision may succeed.
- Preserve reviewer, timestamp and decision history.
- Define whether post-approval edits require reapproval.
- Count visits toward targets only according to agreed policy.
```

5. Security and data integrity

• Verify authentication tokens/sessions server-side for every protected request.

• Enforce role-based access and manager team scope in guards, services and queries.

• Prevent cross-company access and test tenant isolation explicitly.

• Validate request bodies, query parameters, IDs and upload metadata.

• Use safe error messages and never log passwords, tokens or private media URLs.

• Use idempotency keys for retryable visit submissions and safe event processing.

• Use transactions/concurrency controls for approval decisions.

• Use short-lived signed URLs or equivalent controlled access for private attachments.

• Define data retention, access logging, backup encryption and restore procedures.

6. Offline synchronization support

The mobile client can store drafts locally, but the server must validate every submitted record and return a stable ID and authoritative status.

• Accept idempotency keys for operations that may be retried.

• Return clear validation errors and distinguish retryable from permanent failures.

• Return authoritative status after submission and after approval/rejection.

• Define conflict handling for records edited or decided while the device was offline.

• Do not treat a client-provided timestamp as the sole proof of attendance.

7. Backend implementation roadmap

Phase 1 - API foundation

• Create NestJS project, config validation, health endpoint, logging and API versioning.

• Set up PostgreSQL, Prisma schema and migrations.

Phase 2 - Identity and access

• Implement authentication verification and current-user endpoint.

• Add company, role and team scoping; test unauthorized and cross-tenant requests.

Phase 3 - Core field operations

• Implement customers/contacts and attendance policy.

• Implement visit draft, submit, list/detail and private attachment flow.

Phase 4 - Admin workflows

• Implement review queue and approve/reject transitions.

• Add audit history, historical date queries, pagination and filters.

Phase 5 - Targets and notifications

• Define target metrics and period boundaries.

• Implement ranking rules, push notifications and scheduled reminders.

Phase 6 - Production hardening

• Add unit/integration/load/security tests.

• Configure backups, monitoring, secrets, deployment and end-to-end tests.

8. Test plan

Test type	Minimum coverage
Unit	Business rules, validation, target calculations and status transitions
Integration	Database queries, transactions, authentication and API responses
Authorization	Own-data access, manager scope, admin permissions and tenant isolation
Concurrency	Simultaneous approve/reject; only one valid transition succeeds
Idempotency	Repeated submission with same key does not create duplicate visits
Operations	Backup restore, job retry, failed push delivery and monitoring alerts
End-to-end	Mobile submission appears in admin; approval updates status and eligible metrics

9. Integration agreement and handoff

Agree on endpoint paths, request/response types, enum values, timezone handling, authentication headers, error format, pagination, file uploads and API versioning. Maintain a shared API specification and update it whenever the contract changes.

Backend-ready means: migrations are reproducible, APIs are documented and tested, permissions are enforced server-side, critical workflows are transactional, and the frontend can integrate without relying on undocumented behaviour.